

对象特性

构造函数与析构函数

```
#include<iostream>
using namespace std;

class person{
public:
    //构造函数
    //没有返回值，不用写void
    //函数名与类名相同
    //构造函数可以有参数，可以发生重载
    //创建对象p时，构造函数会自动调用，而且只调用一次
    person()
    {
        cout<<"构造函数"<<endl;
    }

    //析构函数
    //没有返回值，不用写void
    //函数名与类名相同，前面加 ~
    //析构函数不可以有参数，不可以发生重载
    //对象在销毁前，会自动调用析构函数，而且只调用一次
    ~person()
    {
        cout<<"析构函数"<<endl;
    }
};

void test1()
{
    person p; //在栈上的数据，在test1执行完成后，释放这个对象
}

int main()
{
    test1();

    return 0;
}
```

构造函数的分类以及调用

```
#include<iostream>
using namespace std;

class person{
```

```

public:
    person()
    {
        cout<<"无参数构造函数"<<endl;
    }

    person(int a)
    {
        age=a;
        cout<<"有参数构造函数"<<endl;
    }

    person(const person &p)//1.拷贝构造函数拷贝的是一个与自己相同数据类型的对象。2.用引用的
    方式传递。3.不能更改传入的参数（const）
    {
        age=p.age;
        cout<<"拷贝构造函数"<<endl;
    }

    ~person()
    {
        cout<<"析构函数"<<endl;
    }
    int age;

};

//调用
void test1()
{
    //括号法
    person p1; //无参数
    //调用无参数时不要加(), 不然编译器会认为是函数声明: person p1();
    person p2(10); //有参数
    person p3(p2);
    cout<<p2.age<<endl;
    cout<<p3.age<<endl;

    // //显示法
    // person p1;
    // person p2=person(10);
    // person p3=person(p2);
    //
    // person(10); //匿名对象 特点: 当前行执行结束后, 系统会立即回收掉匿名对象
    //
    // //隐式转换法
    // person p4=10; //person p4=person(10); 有参构造
    // person p5=p4; //person p5=person(p4); 拷贝构造
}

int main()
{
    test1();
}

```

```
    return 0;
}
```

构造函数调用规则

```
#include<iostream>
using namespace std;
//默认情况下，c++编译器至少给一个类添加三个函数：
//1.默认无参构造函数（函数体为空）
//2.默认析构函数（函数体为空）
//3.默认拷贝构造函数（对属性的值进行拷贝）

//如果用户定义有参构造函数，c++不再提供默认无参构造，但是会提供默认拷贝构造
//如果用户定义拷贝构造函数，c++不会再提供其他构造函数
//默认析构函数一直都有

class person{
public:
    int age;
};

void test()
{
    person p1;
    person p2(p1);
}

int main()
{
    test();

    return 0;
}
```

拷贝构造函数

```
#include<iostream>
using namespace std;
//调用拷贝构造函数三种场景
//1.使用一个已经创建好的对象来初始化一个新对象
//2.值传递的方式给函数参数传值
//3.以值传递方式返回局部对象

class person{
public:
    person()
    {
        cout<<"无参数构造函数"<<endl;
    }
}
```

```

    }

    person(int age)
    {
        m_age=age;
        cout<<"有参数构造函数"<<endl;
    }

    person(const person &p)
    {
        m_age=p.m_age;
        cout<<"拷贝构造函数"<<endl;
    }

    ~person()
    {
        cout<<"析构函数"<<endl;
    }

    int m_age;
};

void test1()
{
    person p1(10);
    person p2(p1);
    cout<<p2.m_age<<endl;
}

void dowork(person p)//值传递
{
    //...
}

void test2()
{
    person p;
    dowork(p);//复制出一个副本传给dowork函数，相当于拷贝构造函数 person p(p);
}

person dowork2()
{
    person p1;
    return p1;//根据p1创建新的对象返回
}

void test3()
{
    person p=dowork2();
}

int main()
{
    test1();

```

```
test2();  
return 0;  
}
```

深拷贝与浅拷贝

```
#include<iostream>  
using namespace std;  
  
//浅拷贝：简单的赋值拷贝操作（编译器默认操作）  
//深拷贝：在堆区重新申请空间，进行拷贝操作（new）  
  
class person{  
  
public:  
    person()  
    {  
        cout<<"无参数构造函数"<<endl;  
    }  
  
    person(int age,double height)  
    {  
        m_age=age;  
        m_height=new double(height);  
        cout<<"有参数构造函数"<<endl;  
    }  
  
    person(const person &p)  
    {  
        m_age=p.m_age;//浅拷贝  
        //m_height=p.m_height;浅拷贝  
        m_height=new double(*p.m_height);//深拷贝  
        //如果不采用深拷贝，会释放两次，不合规则，报错  
        cout<<"拷贝构造函数"<<endl;  
    }  
  
    ~person()//析构函数用来释放堆区申请的内存空间  
    {  
        if(m_height!=NULL)//NULL必须大写  
        {  
            delete m_height;//在堆区申请的内训空间需要手动释放  
            m_height=NULL;//防止野指针出现  
        }  
        cout<<"析构函数"<<endl;  
    }  
  
    int m_age;  
    double *m_height;  
};  
  
void test()//栈中先进后出  
{  
    person p1(22,170);
```

```

        person p2(p1);
        cout<<p1.m_age<<" "<<*p1.m_height<<endl;
        cout<<p2.m_age<<" "<<*p2.m_height<<endl;
    }

    int main()
    {
        test();

        return 0;
    }

```

初始化列表

```

#include<iostream>
using namespace std;
//初始化列表语法: 构造函数():属性1(值1),属性2(值2),...{}
class person{
public:
    // person():m_a(10),m_b(20),m_c(30){
    // }
    person(int a,int b,int c):m_a(a),m_b(b),m_c(c){

    }

    int m_a;
    int m_b;
    int m_c;
};

void test()
{
    person p(20,30,40);
    cout<<p.m_a<<endl;
    cout<<p.m_b<<endl;
    cout<<p.m_c<<endl;
}

int main()
{
    test();
    return 0;
}

```

类对象作为类成员

```

#include<iostream>
#include<string>
using namespace std;

```

```

class phone{
public:
    phone(string pname)
    {
        m_pname=pname;
    }

    string m_pname;
};

class person{
public:
    person(string name,string pname):m_name(name),m_phone(pname)
    {
        //当其他类对象作为本类成员，构造时先构造其他类对象，再构造本类
        //析构顺序与构造顺序相反
    }

    string m_name;
    phone m_phone;

};

void test1()
{
    person p("zq","ipone 15 pro max");
    cout<<p.m_name<<"'s phone is "<<p.m_phone.m_pname<<endl;
}

int main()
{
    test1();

    return 0;
}

```

静态成员

```

#include<iostream>
using namespace std;

class person{
public:
    static int m_a;
    //静态成员变量也是有访问权限的
private:
    static int m_b;//类内可以访问，类外访问不了
};

//静态成员变量：类内声明，类外初始化（必须指明是哪个类）
int person::m_a=100;
int person::m_b=300;

```

```

void test1()
{
    person p1;
    cout<<p1.m_a<<endl;//100
    person p2;
    p2.m_a=200;
    cout<<p1.m_a<<endl;//200
    //静态成员变量不属于某个对象，所有对象都共享同一份数据
    //cout<<person::m_b<<endl; 类外不可以访问
}

void test2()
{
    //静态成员变量有两种访问方式：
    //1.通过对象进行访问
    // person p;
    // cout<<p.m_a<<endl;
    //2.通过类名进行访问
    cout<<person::m_a<<endl;
}

int main()
{
    test1();
    test2();

    return 0;
}

```

静态成员变量

```

#include<iostream>
using namespace std;

class person{
public:
    static int m_a;
    //静态成员变量也是有访问权限的
private:
    static int m_b;//类内可以访问，类外访问不了
};

//静态成员变量：类内声明，类外初始化（必须指明是哪个类）
int person::m_a=100;
int person::m_b=300;

void test1()
{
    person p1;
    cout<<p1.m_a<<endl;//100
    person p2;
    p2.m_a=200;
}

```

```

        cout<<p1.m_a<<endl;//200
        //静态成员变量不属于某个对象，所有对象都共享同一份数据
        //cout<<person::m_b<<endl;
    }

    void test2()
    {
        //静态成员变量有两种访问方式：
        //1.通过对象进行享访问
        // person p;
        // cout<<p.m_a<<endl;
        //2.通过类名进行访问
        cout<<person::m_a<<endl;
    }

    int main()
    {
        test1();
        test2();

        return 0;
    }

```

静态成员函数

```

#include<iostream>
using namespace std;
//静态成员函数：
//1.所有对象共享同一个函数
//2.静态成员函数只能访问静态成员变量
class person{
public:
    static void func()
    {
        m_a=100;//静态成员函数只能访问静态成员变量
        //m_b=100;静态成员函数不能访问非静态成员变量
        cout<<"use static void func"<<endl;
    }
    static int m_a;
    //int m_b;
private: //只能类内访问，不能类外访问
    static void func2()
    {
        cout<<"use static void func2"<<endl;
    }
};//函数不需要类外初始化

int person::m_a=0;

//两种访问方式
void test1()
{

```

```

    //通过对象访问
    person p;
    p.func();
    //通过类名访问
    person::func();
    //person::func()2;静态成员函数有访问权限
}

int main()
{
    test1();

    return 0;
}

```

成员变量和成员函数分开存储

```

#include<iostream>
using namespace std;

class person1{

};

class person2{

    int m_a;

};

class person3{
    int m_a; //非静态成员函数属于类的对象上
    static int m_b; //静态成员变量不属于类的对象上
    void func1() //非静态成员函数不属于类的对象上
    {
        //...
    }
    static void func2() //静态成员函数不属于类的对象上
    {
        //...
    }
};
int person3::m_b=100;

void test1()
{
    person1 p;
    cout<<sizeof(p)<<endl; //空对象占用内存空间为 1个字节
    //每个空对象也应该有独一无二的内存地址
}

void test2()

```

```

{
    person2 p;
    cout<<sizeof(p)<<endl;
}

void test3()
{
    person3 p;
    cout<<sizeof(p)<<endl;
}

int main()
{
    test1();//1
    test2();//4
    test3();//4
    return 0;
}

```

this指针

```

#include<iostream>
using namespace std;

class person{
public:
    person(int age)
    {
        this->age=age;//this指针指向 被调用的成员函数 所属的对象
    }

    void personAdd1(person &p)
    {
        this->age += p.age;
    }

    //如果 personAdd2 返回的是值而不是引用，那么每次调用都会返回一个新的临时对象（p3`），则链
    式调用无法实现
    person& personAdd2(person &p)
    {
        this->age += p.age;
        return *this;//this指向p3的指针，*this就是p3这个对象的本体
    }

    int age;
};

void test1()
{
    person p1(18);
    cout<<p1.age<<endl;
}

```

```

void test2()
{
    person p1(10);
    person p2(10);
    p2.personAdd1(p1);
    cout<<p2.age<<endl;

    person p3(10);
    p3.personAdd2(p2).personAdd2(p2).personAdd2(p2); //链式编程思想
    cout<<p3.age<<endl;
}

int main()
{
    //解决名称冲突
    test1();
    test2();

    //返回对象本身用*this

    return 0;
}

```

空指针访问成员函数

```

#include<iostream>
using namespace std;

class person{
public:
    void showcclassname()
    {
        cout<<"this is person class"<<endl;
    }

    void showpersonage() //报错原因是传入的指针为空
    {
        if(this == NULL)
        {
            return;
        }
        cout<<m_age<<endl; //这个地方m_age相当于this->m_age
    }

    int m_age;
};

void test1()
{
    person *p=NULL;

    p->showclassname();
}

```

```

        p->showpersonage();
    }

    int main()
    {
        test1();

        return 0;
    }

```

const修饰成员函数

```

#include<iostream>
using namespace std;

class person{
public:
    void showperson() const
    {
        //this指针的本质是指针常量，指针的指向是不可以修改的
        //this->m_age=100;//在成员函数后加上const后，this指针指向的值也不可以修改
        //this->m_age=100;相当于m_age=100;

        this->m_score=200;
    }
    int m_age;

    void func()
    {
        m_age=100;
    }

    mutable int m_score;//加上关键字mutable后，即使在常量函数中也可以修改这个值
};

void test1()
{
    person p;
    p.showperson();
}

void test2()
{
    const person p2;//对象前面加上const变为常对象
    //p2.m_age=100;
    p2.m_score=100; //加上 mutable关键字后，在常对象下也可以修改

    //常对象只能调用常函数
    p2.showperson();
    //p2.func();如果常对象可以调用非常函数，那么变相修改了m_age，矛盾了
}

```

```
int main()
{
    test1();
    test2();
    return 0;
}
```